

第 13 章

直方图处理

直方图是图像处理过程中的一种非常重要的分析工具。直方图从图像内部灰度级的角度对图像进行表述，包含十分丰富而重要的信息。从直方图的角度对图像进行处理，可以达到增强图像显示效果的目的。

13.1 直方图的含义

从统计的角度讲，直方图是图像内灰度值的统计特性与图像灰度值之间的函数，直方图统计图像内各个灰度级出现的次数。从直方图的图形上观察，横坐标是图像中各像素点的灰度级，纵坐标是具有该灰度级（像素值）的像素个数。

例如，有一幅图像如图 13-1 所示。该图中只有 9 个像素点，存在 1、2、3、4、5，共 5 个灰度级。

1	1	3
2	3	5
5	1	4

图 13-1 图像示例

统计各个灰度级出现的次数，如表 13-1 所示。

表 13-1 图像中各灰度级出现的次数

灰度级	1	2	3	4	5
出现的次数	3	1	2	1	2

在绘制直方图时，将灰度级作为 x 轴处理，该灰度级出现的次数作为 y 轴处理，则可知：

- x 轴的数据为 $x=[1\ 2\ 3\ 4\ 5]$ 。
- y 轴的数据为 $y=[3\ 1\ 2\ 1\ 2]$ 。

根据上述关系，可以绘制出如图 13-2 所示的折线图（左图）和直方图（右图）。一般情况下，我们把左侧的直线图和右侧直方图都称为直方图。



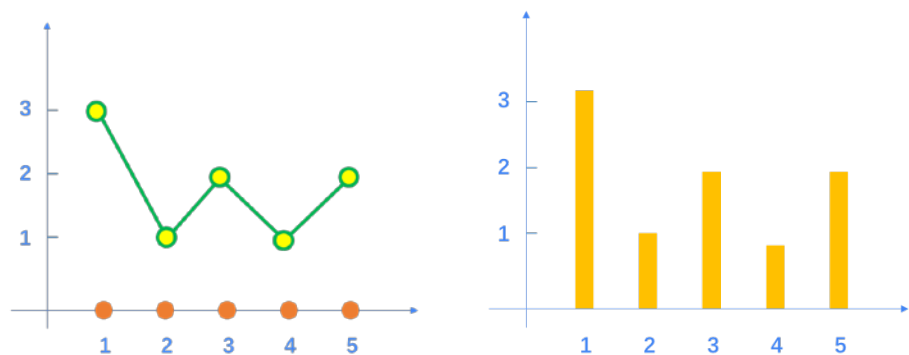


图 13-2 直方图

在实际处理中,图像直方图的 x 轴区间一般是 $[0, 255]$, 对应的是 8 位位图的 256 个灰度级; y 轴对应的是具有相应灰度级的像素点的个数。

例如在图 13-3 中, 上图是一张图像, 下图则是其对应的直方图。图中圆点表示这些像素点会被统计到对应的灰度级上。

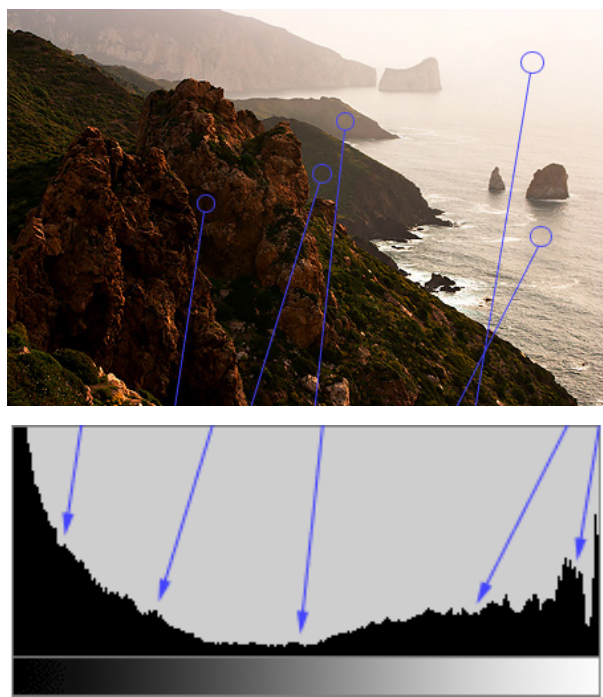
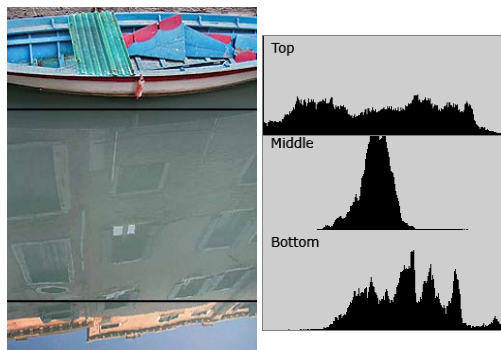


图 13-3 直方图示例¹之一

虽然 8 位的图像都具有 256 个灰度级 (每一个像素可以有 256 个灰度值), 但是属于不同灰度级的像素数量是很不一样的。例如图 13-4, 从图中可以看出, 图像的不同部分直方图是不一样的。

1 来自 <http://www.cambridgeincolour.com/tutorials/histograms1.htm>。

图 13-4 直方图示例¹之二

有时为了便于表示，也会采用归一化直方图。在归一化直方图中，x 轴仍然表示灰度级；y 轴不再表示灰度级出现的次数，而是灰度级出现的频率。

例如，针对图 13-1，统计各个灰度级出现的频率：

灰度级出现的频率 = 灰度级出现的次数/总像素数

在图 13-1 中共有 9 个像素，所以统计结果如表 13-2 所示。

表 13-2 灰度级出现的频率

灰度级	1	2	3	4	5
出现的频率	3/9	1/9	2/9	1/9	2/9

在归一化直方图中，各个灰度级出现的频率之和为 1。例如，本例中：

$$\frac{3}{9} + \frac{1}{9} + \frac{2}{9} + \frac{1}{9} + \frac{2}{9} = 1$$

在绘制直方图时，将灰度级作为 x 轴数据处理，将其出现的频率作为 y 轴数据处理，则可知：

- x 轴的数据为 $x=[1\ 2\ 3\ 4\ 5]$
- y 轴的数据为 $y=[3/9\ 1/9\ 2/9\ 1/9\ 2/9]$

根据上述关系，可以绘制出如图 13-5 所示的归一化直方图。对比图 13-4 与图 13-5，可以看到，归一化直方图与直方图在外观上是一致的，只是 y 轴的标签不同而已。本例中，在直方图内，y 轴显示的标签是 1、2、3；在归一化直方图中，y 轴显示的标签是 1/9、2/9、3/9。

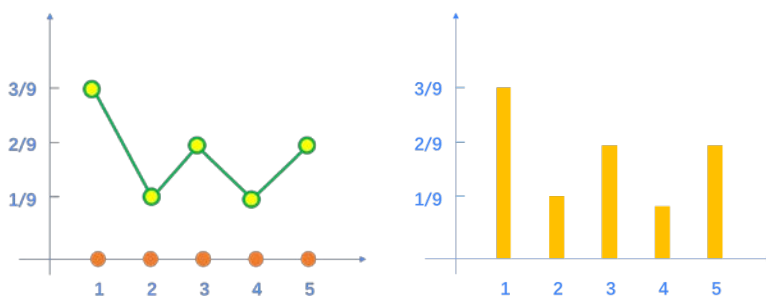


图 13-5 归一化直方图

1 来自 <http://www.cambridgeincolour.com/tutorials/histograms1.htm>。

在 OpenCV 的官网上，特别提出了要注意三个概念：DIMS、BINS、RANGE。

- **DIMS**: 表示在绘制直方图时，收集的参数的数量。一般情况下，直方图中收集的数据只有一种，就是灰度级。因此，该值为 1。
- **RANGE**: 表示要统计的灰度级范围，一般为[0, 255]。0 对应的是黑色，255 对应的是白色。
- **BINS**: 参数子集的数目。在处理数据的过程中，有时需要将众多的数据划分为若干组，再进行分析。

例如，针对图 13-1 中的灰度级，你可能希望将两个像素值作为一组讨论。这样，整个灰度级被划分为三组，具体为 { 1,2 }, { 3,4 }, { 5 } }。图 13-6 所示的是划分前后的直方图情况。

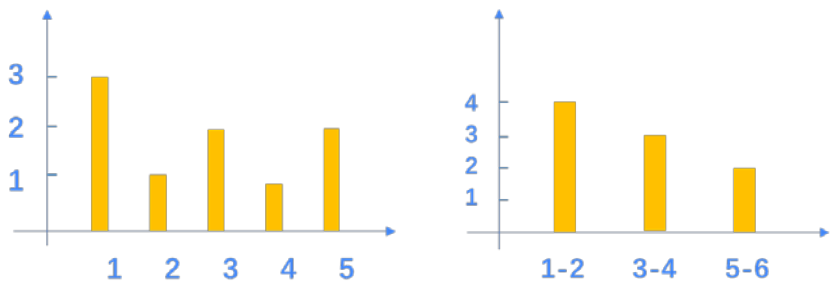


图 13-6 灰度级分组前后的直方图

也可以按照上述方式对灰度图像进行划分。例如，在灰度图像中，将[0, 255]区间内的 256 个灰度级，按照每 16 个像素一组划分为子集：

$$[0, 255] = [0, 15] \cup [16, 31] \cup \dots \cup [240, 255]$$

按照上述方式，整个灰度级范围可以划分为 16 个子集，具体为：

$$\text{整个灰度级范围} = \text{bin1} \cup \text{bin2} \cup \dots \cup \text{bin16}$$

子集划分完以后，某灰度图像生成的直方图如图 13-7 所示（图中的 b1 代表 bin1，b2 代表 bin2，以此类推）。

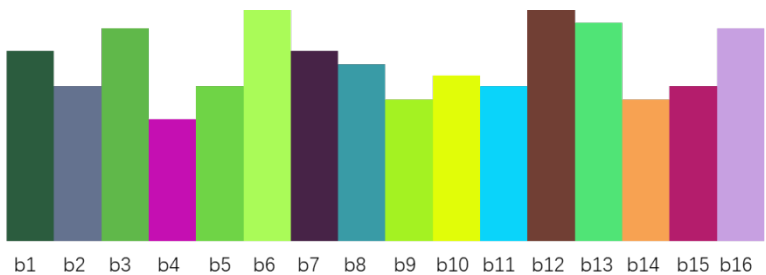


图 13-7 直方图示例

下面讨论 BINS 的值：

- 针对图 13-1，在原始图像中，共有 5 个灰度级，其 BINS 值为 5。在以 2 个灰度级为一个小组划分子集后，得到 3 个子集，其 BINS 值为 3。

- 针对灰度图像，灰度级区间为[0, 255]，共有 256 个灰度级，其 BINS 值为 256；在以 16 个灰度级为一个小组划分子集后，其 BINS 值为 16。

13.2 绘制直方图

Python 的模块 `matplotlib.pyplot` 中的 `hist()` 函数能够方便地绘制直方图，我们通常采用该函数直接绘制直方图。除此以外，OpenCV 中的 `cv2.calcHist()` 函数能够计算统计直方图，还可以在此基础上绘制图像的直方图。下面分别讨论这两种方式。

13.2.1 使用 Numpy 绘制直方图

模块 `matplotlib.pyplot` 提供了一个类似于 MATLAB 绘图方式的框架，可以使用其中的 `matplotlib.pyplot.hist()` 函数（以下简称为 `hist()` 函数）来绘制直方图。

此函数的作用是根据数据源和灰度级分组绘制直方图。其基本语法格式为：

```
matplotlib.pyplot.hist(X, BINS)
```

其中两个参数的含义如下：

- **X**：数据源，必须是一维的。图像通常是二维的，需要使用 `ravel()` 函数将图像处理为一维数据源以后，再作为参数使用。
- **BINS**：BINS 的具体值，表示灰度级的分组情况。

函数 `ravel()` 的作用是将二维数组降维成一维数组。例如，有图像 `a`，其值为：

21	34	63
142	231	59
67	138	74

使用函数 `ravel()` 对 `a` 进行处理：

```
b = a.ravel()
```

可以得到 `b` 为：

21	34	63	142	231	59	67	138	74
----	----	----	-----	-----	----	----	-----	----

【例 13.1】 使用 `hist()` 函数绘制一幅图像的直方图。

根据题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
o=cv2.imread("image\\boat.jpg")
cv2.imshow("original",o)
plt.hist(o.ravel(),256)
cv2.waitKey()
cv2.destroyAllWindows()
```



运行上述程序后，得到如图 13-8 所示的结果。其中左图是原始图像，右图是其对应的直方图。

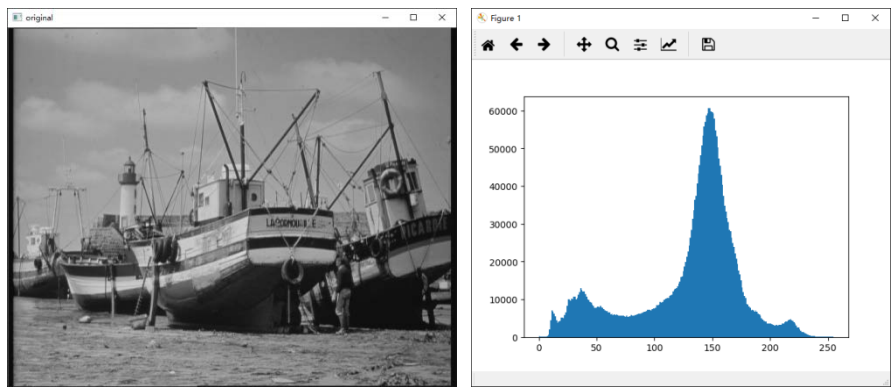


图 13-8 【例 13.1】程序的运行结果

【例 13.2】使用函数 `hist()` 将一幅图像的灰度级划分为 16 组后，绘制该图像的直方图。

将灰度级划分为 16 组，即将灰度级划分为 16 个子集，对应的 BINS 值为 16。按照题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
o=cv2.imread("image\\boat.bmp")
plt.hist(o.ravel(),16)
```

运行上述程序，结果如图 13-9 所示，从图中可以看到，整个灰度级被划分为 16 个子集。

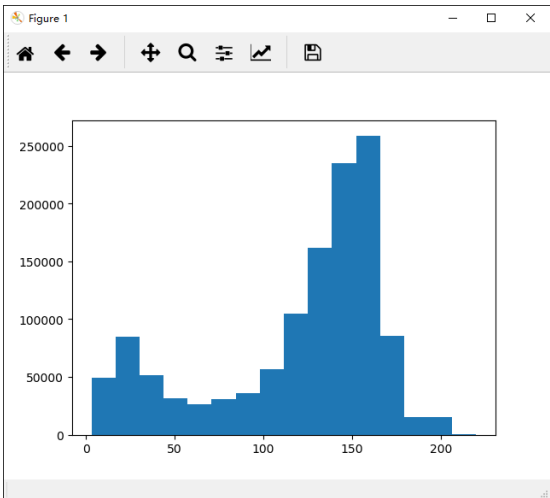


图 13-9 BINS 值为 16 的直方图

13.2.2 使用OpenCV绘制直方图

OpenCV 提供了函数 `cv2.calcHist()` 用来计算图像的统计直方图，该函数能统计各个灰度级的像素点个数。利用 `matplotlib.pyplot` 模块中的 `plot()` 函数，可以将函数 `cv2.calcHist()` 的统计结

果绘制成直方图。

1. 用cv2.calcHist()函数统计图像直方图信息

函数 `cv2.calcHist()` 用于统计图像直方图信息，其语法格式为：

```
hist = cv2.calcHist( images, channels, mask, histSize, ranges, accumulate )
```

函数中返回值及参数的含义为：

- **hist**: 返回的统计直方图，是一个一维数组，数组内的元素是各个灰度级的像素个数。
- **images**: 原始图像，该图像需要使用 “[]” 括起来。
- **channels**: 指定通道编号。通道编号需要用 “[]” 括起来，如果输入图像是单通道灰度图像，该参数的值就是 [0]。对于彩色图像，它的值可以是 [0]、[1]、[2]，分别对应通道 B、G、R。
- **mask**: 掩模图像。当统计整幅图像的直方图时，将这个值设为 `None`。当统计图像某一部分的直方图时，需要用到掩模图像。
- **histSize**: BINS 的值，该值需要用 “[]” 括起来。例如，BINS 的值是 256，需要使用 “[256]” 作为此参数值。
- **ranges**: 即像素值范围。例如，8 位灰度图像的像素值范围是 [0, 255]。
- **accumulate**: 累计（累积、叠加）标识，默认值为 `False`。如果被设置为 `True`，则直方图在开始计算时不会被清零，计算的是多个直方图的累积结果，用于对一组图像计算直方图。该参数允许从多个对象中计算单个直方图，或者实时更新直方图。该参数是可选的，一般情况下不需要设置。

【例 13.3】 使用 `cv2.calcHist()` 函数计算一幅图像的统计直方图结果，并观察得到的统计直方图信息。

根据题目的要求，编写代码如下：

```
import cv2
import numpy as np
img=cv2.imread("image\\boat.jpg")
hist = cv2.calcHist([img],[0],None,[256],[0,255])
print(type(hist))
print(hist.shape)
print(hist.size)
print(hist)
```

需要注意，在本例的 `cv2.calcHist()` 函数中：

- 第 1 个参数 “[img]” 表示要绘制直方图的原始图像，是使用 “[]” 括起来的。
- 第 2 个参数表示要统计哪个通道的直方图信息。本例中读取的 `img` 是灰度图像，所以使用 “[0]” 来表示。
- 第 3 个参数是掩模图像，在本例中的值为 “None”，表示计算整幅图像的直方图。
- 第 4 个参数 “[256]” 表示 BINS 的值是 256。
- 第 5 个参数 “[0, 255]” 表示灰度级的范围是 [0, 255]。



程序运行结果如图 13-10 所示。

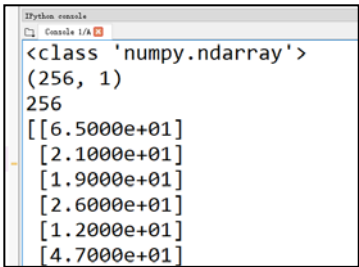


图 13-10 【例 13.3】程序的运行结果

从图 13-10 中可以看到，函数 `cv2.calcHist()` 返回值的数据类型为“`ndarray`”。该数据的 `shape` 为 `(256,1)`，说明其有 256 行 1 列。该数据的 `size` 为 256，说明有 256 个元素，分别对应着 256 个灰度级在图像内出现的次数。

图 13-10 所示的程序运行结果的下半部分是 `hist` 内的部分数据，限于篇幅，这里仅截取了 256 条数据中的前 6 条。这 6 条数据，对应着灰度级为 0~5 的像素点出现的次数。

2. `plot()`函数的使用

使用 `matplotlib.pyplot` 模块内的 `plot()` 函数，可以将函数 `cv2.calcHist()` 的返回值绘制为图像直方图。下面通过三个例子来学习 `plot()` 函数的基本使用方法。

【例 13.4】 将给定的 `x = [0,1,2,3,4,5,6]`，`y = [0.3,0.4,2,5,3,4.5,4]`，使用 `plot()` 函数绘制出来。

根据题目的要求，编写代码如下：

```
import matplotlib.pyplot as plt
x = [0,1,2,3,4,5,6]
y = [0.3,0.4,2,5,3,4.5,4]
plt.plot(x,y)
```

程序的运行结果如图 13-11 所示。图中 `x` 轴由 `x = [0, 1, 2, 3, 4, 5, 6]` 指定，`y` 轴由 `y = [0.3, 0.4, 2, 5, 3, 4.5, 4]` 指定。

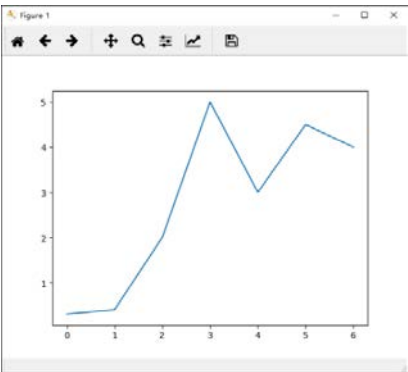


图 13-11 【例 13.4】程序的运行结果

【例 13.5】 给定 `y = [0.3, 0.4, 2, 5, 3, 4.5, 4]`，使用 `plot()` 函数将其绘制出来，观察绘制结果。

根据题目的要求，编写如下代码：


```
import matplotlib.pyplot as plt
y = [0.3,0.4,2,5,3,4.5,4]
plt.plot(y)
```

执行上述程序，运行结果如图 13-12 所示。从图中可以看出，在使用 plot()函数时，如果仅仅指定一个参数，则其对应 x 轴的值默认是一个自然数序列 $x=[0, 1, \dots, n-1, n]$ 。自然序列 x 的长度与 y 的长度保持一致。

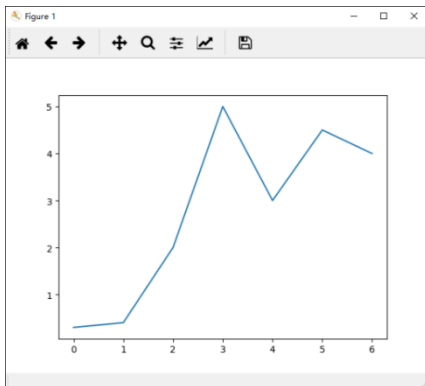


图 13-12 【例 13.5】程序的运行结果

【例 13.6】使用 plot()函数将两组不同的值 $a=[0.3, 0.4, 2, 5, 3, 4.5, 4]$ ， $b=[3, 5, 1, 2, 1, 5, 3]$ 以不同的颜色绘制出来。

根据题目要求，编写代码如下：

```
import matplotlib.pyplot as plt
a = [0.3,0.4,2,5,3,4.5,4]
b=[3,5,1,2,1,5,3]
plt.plot(a,color='r')
plt.plot(b,color='g')
```

上述代码中：

- color='r'表示绘图曲线的 color 属性是“red”，即绘图颜色是红色的。
- color='g'表示绘图曲线的 color 属性是“green”，即绘图颜色是绿色的。

在绘图时，可以根据需要将 color 属性设置为不同颜色。执行上述程序，绘图结果如图 13-13 所示。因为印刷条件限制无法显示彩色的结果，大家可以自行运行该程序，观察运行效果。

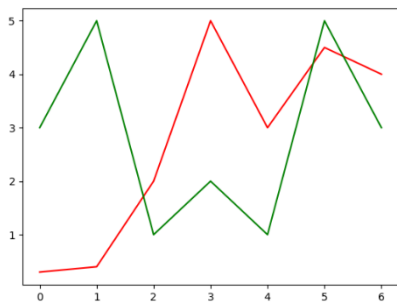


图 13-13 【例 13.6】程序的运行结果

3. 绘制统计直方图

熟悉函数 `plot()` 的使用后，就可以使用它将函数 `cv2.calcHist()` 的返回值绘制为直方图了。

【例 13.7】 使用函数 `plot()` 将函数 `cv2.calcHist()` 的返回值绘制为直方图。

根据题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
o=cv2.imread("image\\boatGray.bmp")
histb = cv2.calcHist([o],[0],None,[256],[0,255])
plt.plot(histb,color='b')
plt.show()
```

运行上述程序，得到如图 13-14 所示直方图。

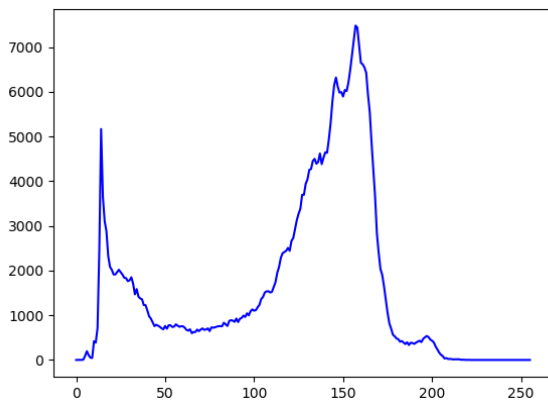


图 13-14 使用 `plot()` 绘制的直方图

本例首先通过函数 `cv2.calcHist()` 得到图像的统计直方图数据，然后利用函数 `plot()` 将这些数据绘制出来。

【例 13.8】 使用函数 `plot()` 和函数 `cv2.calcHist()`，将彩色图像各个通道的直方图绘制在一个窗口内。

根据题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
o=cv2.imread("image\\girl.bmp")
histb = cv2.calcHist([o],[0],None,[256],[0,255])
histg = cv2.calcHist([o],[1],None,[256],[0,255])
histr = cv2.calcHist([o],[2],None,[256],[0,255])
plt.plot(histb,color='b')
plt.plot(histg,color='g')
plt.plot(histr,color='r')
plt.show()
```

运行上述程序，得到如图 13-15 所示直方图。

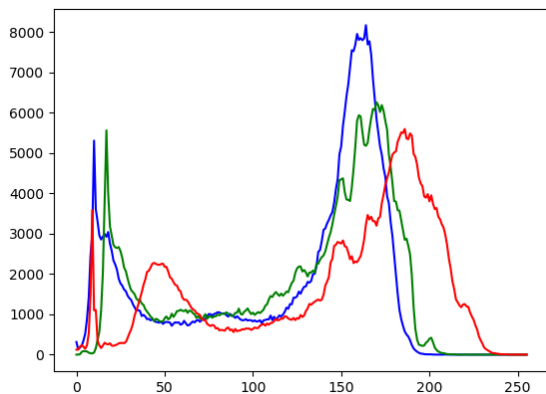


图 13-15 绘制彩色图像各个通道的直方图

本例先通过函数 `cv2.calcHist()` 分别得到 [0]、[1]、[2] 三个通道（即 B、G、R 三个通道）的统计直方图数据，然后再通过 `plot()` 函数将这些数据绘制成直方图。

13.2.3 使用掩模绘制直方图

在函数 `cv2.calcHist()` 中，参数 `mask` 用于标识是否使用掩模图像。当使用掩模图像获取直方图时，仅获取掩模参数 `mask` 指定区域的直方图。

1. 掩模处理是怎么回事

首先，我们来看掩模处理的基本形式。在图 13-16 中，左图是原始图像 O，中间的是掩模图像 M，右图是使用掩模图像 M 对原始图像 O 进行掩模运算的结果图像 R。



图 13-16 掩模处理

为了方便理解，可以将掩模图像看作一块玻璃板，玻璃板上的白色区域是透明的，黑色区域是不透明的。掩模运算就是将该玻璃板覆盖在原始图像上，透过玻璃板显示出来的部分就是掩模运算的结果图像。

在掩模运算中，使用了“与”和“或”运算，我们用图 13-17 所示的电路图进行说明。

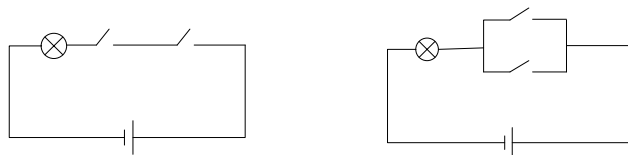


图 13-17 “与”“或”运算的电路示意图

- 与运算（AND）和串联电路是对应的。在图 13-17 的左图所示的串联电路中，只有当两个开关都闭合时，电路才是连通的。对应数值运算上，只有参与运算的数值都是 1 时，与运算结果才是 1，如表 13-3 所示。

表 13-3 与运算

运算值 1	运算值 2	结果
0	0	0
0	1	0
1	0	0
1	1	1

- 或运算（OR）和并联电路是对应的。在图 13-17 的右图所示的并联电路中，任意一个开关闭合，电路都是连通的。对应数值运算上，参与运算的数值只要有一个值是 1 时，或运算结果就是 1，如表 13-4 所示。

表 13-4 或运算

运算值 1	运算值 2	结果
0	0	0
0	1	1
1	0	1
1	1	1

将图 13-18 中左侧的原始图像 O 与中间的掩模图像 M，进行与运算，得到右侧的结果图像 R。经过与运算后，在原始图像 O 中，只有深色背景区域中的值被保留，其余的值都被处理为 0。

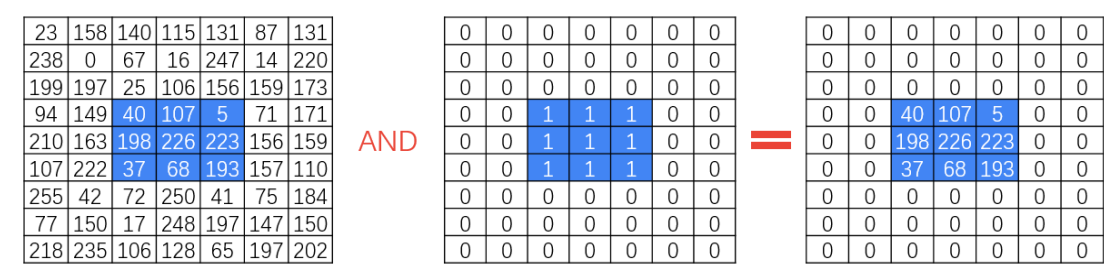


图 13-18 与运算示意图

得到上述运算结果是因为：

- 原始图像 O 的深色背景区域，所对应掩模图像中深色背景区域的值均为 1，对这两个深色背景区域进行与运算时，根据运算规则，原始图像 O 深色背景区域的值被保留。
- 原始图像 O 的白色背景区域，所对应掩模图像中白色背景区域的值均为 0，对这两个白色背景区域进行与运算时，根据运算规则，原始图像 O 白色背景区域的值被处理为 0。

上述与运算是直接对两个数值进行运算的，理解起来比较方便，但是在实际处理中较为烦琐。在实际处理中，通常将掩模图像白色区域的值设置为 255，然后让掩模图像与原始图像进行按位与运算，得到结果图像。下面简单介绍一下按位与运算的基本原理，更多内容请参考第 3 章。

OpenCV 提供了 `bitwise_and` 函数，用于两个像素值之间的按位与运算。按位与运算，是指

将两个像素值按照二进制位的关系，逐位进行与运算。例如，A 的像素值为 35，B 的像素值为 126，将 A 和 B 进行按位与运算，所得结果如表 13-5 所示。在进行按位与运算时，需要先将参与运算的值转换为二进制数，然后将对应位上的值进行与运算。

表 13-5 按位与运算示例

说明	二进制值
A (35)	0010 0011
B (126)	0111 1110
与运算的结果 (34)	0010 0010

根据与运算的规则，任意一个数值与数值 1 进行与运算，结果等于其自身的值。因此，任意一个 8 位像素值与二进制数“1111 1111”进行按位与运算，得到的都是自身的值。二进制值“1111 1111”对应的十进制值是“255”，所以任意一个 8 位像素值与“255”进行按位与运算，得到的都是自身原来的值。例如，像素点 X 的值为 58，像素点 Y 的值为 135，将这两个值与 255 进行按位与运算后，得到的是它们自身的值，如表 13-6 所示。

表 13-6 按位与运算示例

说明	像素点 X (58)	像素点 Y (135)
像素点对应的二进制值	0011 1010	1000 0111
255 的二进制值	1111 1111	1111 1111
按位与运算的结果	0011 1010	1000 0111
十进制值	58	135
是否与原值相等	相等	相等

当然，任意数值与数值 0 进行与的运算结果都是 0，如表 13-7 所示。

表 13-7 按位与运算示例

说明	像素点 X (58)	像素点 Y (135)
像素点对应的二进制值	0011 1010	1000 0111
0 的二进制值	0000 0000	0000 0000
按位与运算的结果	0000 0000	0000 0000
十进制值	0	0

根据上述分析，可以将掩模图像的白色背景部分设置为 255，黑色背景部分设置为 0，如图 13-19 所示。

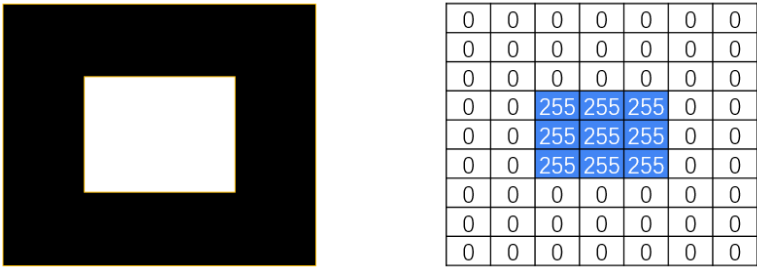


图 13-19 掩模图像的构成

综上所述，将一幅图像与如图 13-19 所示的掩模图像进行按位与运算后，得到的结果图像分为以下两部分。

第 1 部分：与掩模图像中黑色背景位置对应的部分，该部分的像素值都被置为零。

第 2 部分：与掩模图像中白色背景位置对应的部分，该部分的像素保留原有值。

例如，在图 13-16 中，最左侧的图像与中间掩模图像进行按位与运算，得到了最右侧的运算结果。原始图像中与掩模图像白色背景对应的部分被保留，与黑色背景对应的部分被处理为零值（黑色）。

当在一个函数 F 中使用了掩模时，就意味着要先将参与运算的图像与掩模图像进行按位与运算，得到掩模运算结果（处理过程如图 13-16 所示）。然后，再将掩模运算结果，作为函数 F 的参数完成函数 F 所指定的运算。例如，有函数 F 、原始图像 I 、掩模图像 M 和运算结果 R ，则：

- $R = F(I)$ 表示直接使用函数 F 对图像 I 进行处理。
- $R = F(I, M)$ 表示先将 I 和 M 进行按位与，然后再使用函数 F 对上述按位与的结果进行处理。

上述过程就是掩模处理的原理和方式。不过，在实际使用中我们只需要构造掩模图像就可以了，至于掩模图像如何起作用，并不需要我们的操心，是由函数来实现的。

2. 如何构造掩模图像

在构造掩模图像时，通常先构造一个像素值都是 0 的二维数组，再将数组中指定区域的像素值设定为 255，就得到了掩模图像。下面通过一个例子来说明如何构造掩模图像。

【例 13.9】 构造一个中心点是白色区域的掩模图像。

根据题目的要求，首先使用函数 `np.zeros()` 构造一个像素值都是 0 的二维数组，然后将其中间部分的值设置为 255，代码如下：

```
import cv2
import numpy as np
mask=np.zeros([600,600],np.uint8)
mask[200:400,200:400]=255
cv2.imshow('mask',mask)
cv2.waitKey()
cv2.destroyAllWindows()
```

运行上述程序，生成如图 13-20 所示的掩模图像。

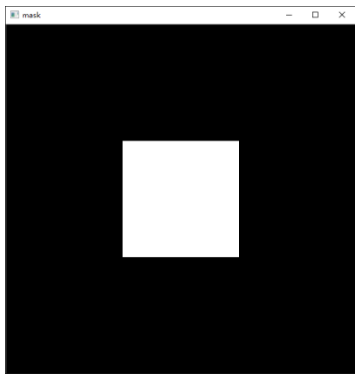


图 13-20 【例 13.9】程序生成的掩模图像

3. 使用掩模绘制直方图

绘制掩模图像时，首先将函数 `cv2.calcHist()` 的 `mask` 参数设置为掩模图像，得到掩模处理的直方图信息，再使用 `plot()` 函数完成直方图的绘制。

回顾一下函数 `cv2.calcHist()` 的语法格式：

```
hist = cv2.calcHist( images, channels, mask, histSize, ranges, accumulate )
```

其中，`mask` 参数就是掩模图像。

下面通过一个例子演示如何绘制掩模结果图像的直方图。

【例 13.10】 演示绘制掩模结果图像的直方图。

根据题目的要求，首先构造一个掩模图像，然后使用函数 `cv2.calcHist()` 计算掩模结果图像的统计直方图信息，最后使用函数 `plot()` 绘制掩模图像的直方图。

掩模图像要保持与原始图像相等的大小，这里使用参数 `image.shape` 表示构造与原始图像等大小的掩模图像。

代码如下：

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
image=cv2.imread("image\\girl.bmp",cv2.IMREAD_GRAYSCALE)
mask=np.zeros(image.shape,np.uint8)
mask[200:400,200:400]=255
histImage=cv2.calcHist([image],[0],None,[256],[0,255])
histMI=cv2.calcHist([image],[0],mask,[256],[0,255])
plt.plot(histImage)
plt.plot(histMI)
```

运行上述程序，生成如图 13-21 所示的直方图。其中，上方的曲线是原始图像的直方图，下方的曲线是掩模结果图像的直方图。

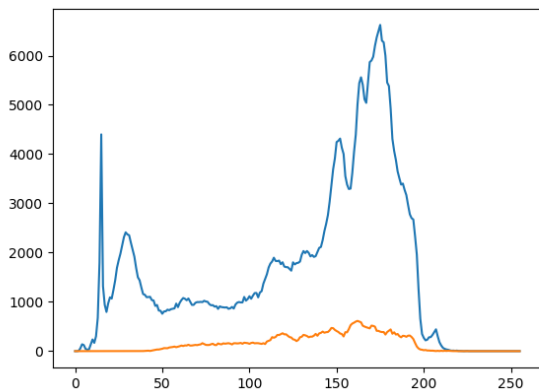


图 13-21 掩模结果图像的直方图

13.3 直方图均衡化

如果一幅图像拥有全部可能的灰度级，并且像素值的灰度均匀分布，那么这幅图像就具有高对比度和多变的灰度色调，灰度级丰富且覆盖范围较大。在外观上，这样的图像具有更丰富的色彩，不会过暗或过亮。

图 13-22 展示了对一幅图像进行直方图均衡化前后的对比，左图是原始图像，比较暗；右图是均衡化后的图像，色彩比较均衡。



图 13-22 直方图均衡化前后图像的显示效果对比

在 OpenCV 的官网上，对图像均衡化（即直方图均衡化）前后的直方图进行了对比，如图 13-23 所示。其中，左图是原始图像的直方图，可以看到灰度级集中在中间，图像中没有较暗和较亮的像素点；右图是对原图均衡化后的直方图，像素分布更均衡。

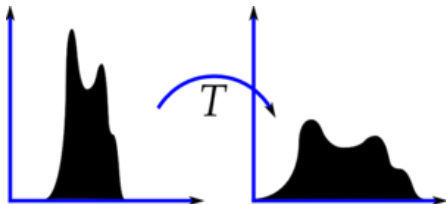


图 13-23 图像均衡化前后的直方图对比

直方图均衡化的主要目的是将原始图像的灰度级均匀地映射到整个灰度级范围内，得到一个灰度级分布均匀的图像。这种均衡化，既实现了灰度值统计上的概率均衡，也实现了人类视觉系统（Human Visual System，HVS）上的视觉均衡。

例如，在某幅图像内仅仅出现了 1、2、3、101、102、103 等 6 个像素值，其分布分别如表 13-8 中的情况 A 和情况 B 所示。

表 13-8 某图像的像素值分布

灰度级(像素值)		1	2	3	101	102	103
像素个数	情况 A	1	1	1	1	1	1
	情况 B	1	1	1	0	0	3

下面分别讨论这两种情况下像素值均衡的情况。

- **情况 A：** 每一个灰度级在图像内出现的次数都是 1，灰度级均匀地映射到当前的灰度级

范围内，所以可以理解为其直方图是均衡的。

- **情况 B:** 灰度级 1、2、3 出现的次数都是 1 次，灰度级 103 出现的次数是 3 次，灰度级 101、102 出现的次数是 0 次。从表面上看，灰度级是不均衡的。但是，从 HVS 的角度来说，人眼的敏感度不足以区分 1 个像素值的差异，即人眼会将灰度级 1、2 和 3 看作是相同的，会将灰度级 101、102 和 103 看作是相同的。也就是说，HVS 会自动地将灰度级划分为两组，灰度级[1, 3]为一组，灰度级[101, 103]为另一组。在整幅图像内，这两组的灰度级出现的次数都是 3 次，概率是相等的。在均衡化处理中，综合考虑了统计概率和 HVS 的均衡。

13.3.1 直方图均衡化原理

直方图均衡化的算法主要包括两个步骤：

- (1) 计算累计直方图。
- (2) 对累计直方图进行区间转换

在此基础上，再利用人眼视觉达到直方图均衡化的目的。

下面我们通过一个例子进行讲解。例如，图像 A 如图 13-24 所示，它是一幅 3 位的位图，即共有 8 (2^3) 个灰度级，有 49 个像素。

0	1	4	1	7	3	3
0	0	4	0	0	1	3
1	2	7	5	7	4	6
0	4	0	1	1	6	6
7	1	2	2	7	3	3
4	5	7	4	2	7	2
0	7	1	5	2	0	1

图 13-24 图像 A

图像 A 共有 8 个灰度级，范围为[0, 7]，计算其统计直方图如表 13-9 所示。

表 13-9 图像 A 的统计直方图

灰度级	0	1	2	3	4	5	6	7
像素个数	9	9	6	5	6	3	3	8

在此基础上，计算归一化统计直方图，计算方式是计算每个像素在图像内出现的概率。出现概率=出现次数/像素总数，用每个灰度级的像素个数除以总的像素个数（49），就得到归一化统计直方图，如表 13-10 所示。



表 13-10 图像 A 的归一化统计直方图

灰度级	0	1	2	3	4	5	6	7
概率	0.18	0.18	0.12	0.10	0.12	0.06	0.06	0.16

接下来，计算累计统计直方图，即计算所有灰度级的累计概率，结果如表 13-11 所示。

表 13-11 图像 A 的累计统计直方图

灰度级	0	1	2	3	4	5	6	7
累计概率	0.18	0.37	0.49	0.59	0.71	0.78	0.84	1.00

在累计直方图的基础上，对原有灰度级空间进行转换。可以在原有范围内对灰度级实现均衡化，也可以在更广泛的灰度空间范围内对灰度级实现均衡化。下面分别介绍这两种形式。

1. 在原有范围内实现均衡化

在原有范围内实现直方图均衡化时，用当前灰度级的累计概率乘以当前灰度级的最大值 7，得到新的灰度级，并作为均衡化的结果。表 13-12 所示的就是计算得到的新灰度级。

表 13-12 计算得到的新灰度级

灰度级	0	1	2	3	4	5	6	7
累计概率	0.18	0.37	0.49	0.59	0.71	0.78	0.84	1.00
均衡化值（新的灰度级）	1	3	3	4	5	5	6	7

根据表 13-9 中各个灰度级的关系，完成均衡化值（新的灰度级）的映射，如表 13-13 所示。此处的对应关系为：

- 原始图像 A 中的灰度级 0，经直方图均衡化后调整为新的灰度级 1（即均衡化值 1）。在原始图像 A 中，灰度级 0 共有 9 个像素点，所以在均衡化后的图像中，灰度级 1 共有 9 个像素点。
- 原始图像 A 中的灰度级 1 和 2 经直方图均衡化后调整为灰度级 3。在原始图像 A 中，灰度级 1 共有 9 个像素点，灰度级 2 共有 6 个像素点，所以在均衡化后的图像中，灰度级 3 共有 $9 + 6 = 15$ 个像素点。
- 原始图像 A 中的灰度级 3 经直方图均衡化后调整为灰度级 4。在原始图像 A 中，灰度级 3 共有 5 个像素点，所以在均衡化后的图像中，灰度级 4 共有 5 个像素点。
- 原始图像 A 中的灰度级 4 和 5 经直方图均衡化后调整为灰度级 5。在原始图像 A 中，灰度级 4 共有 6 个像素点，灰度级 5 共有 3 个像素点，所以在均衡化后的图像中，灰度级 5 共有 $6 + 3 = 9$ 个像素点。
- 原始图像 A 中的灰度级 6 经直方图均衡化后调整为灰度级 6。在原始图像 A 中，灰度级 6 共有 3 个像素点，所以在均衡化后的图像中，灰度级 6 共有 3 个像素点。
- 原始图像 A 中的灰度级 7 经直方图均衡化后调整为灰度级 7。在原始图像 A 中，灰度级 7 共有 8 个像素点，所以在均衡化后的图像中，灰度级 7 共有 8 个像素点。

表 13-13 归一化统计直方图

灰度级	0	1	2	3	4	5	6	7
像素个数	9	9	6	5	6	3	3	8
均衡化值	1	3	3	4	5	5	6	7



在均衡化后的图像中,不存在灰度级 0 和灰度级 2 的像素点。

根据上述关系,可以得到表 13-14。其中列出了直方图均衡化后每个灰度级的像素个数。

表 13-14 均衡化后对应灰度级的像素个数

均衡化后的灰度级	0	1	2	3	4	5	6	7
像素个数	0	9	0	15	5	9	3	8

经过均衡化处理后,灰度级在整个灰度空间内的分布会更均衡。图 13-25 所示的是直方图均衡化前后的对比图。其中,左图是均衡化之前的直方图,右图是均衡化之后的直方图。

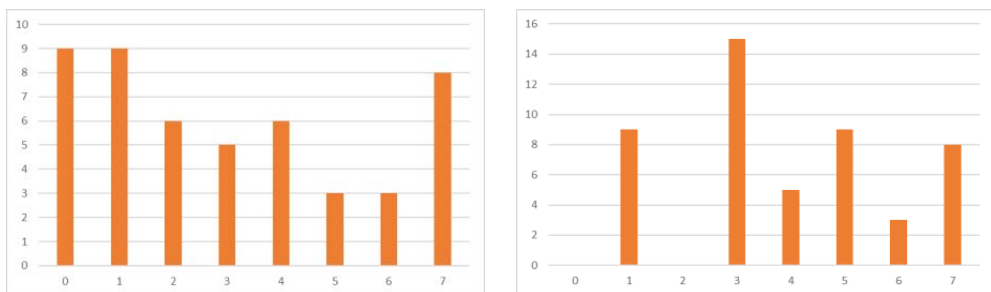


图 13-25 直方图均衡化前后的对比

从图 13-25 中可以看出,右图的灰度级在整个灰度空间内分布更均衡。值得注意的,这里的均衡化是综合考虑了统计概率和 HVS 的结果。

- 在图像 A 中,未进行直方图均衡化之前:灰度级 0 ~ 3 之间的像素个数为 29 个,灰度级 4 ~ 7 之间的像素个数为 20 个。
- 对图像 A 进行直方图均衡化之后:灰度级 0 ~ 3 之间的像素个数为 24 个,灰度级 4 ~ 7 之间的像素个数为 25 个。

通过上述比较,可以看出,直方图均衡化之后图像的灰度级分布更均衡了。

2. 在更广泛的范围内实现均衡化

在更广泛的范围内实现直方图均衡化时,用当前灰度级的累计概率乘以更广泛范围灰度级的最大值,得到新的灰度级,并作为均衡化的结果。例如,要将灰度级空间扩展为[0, 255]共 256 个灰度级,就必须将原灰度级的累计概率乘以 255,得到新的灰度级。表 13-15 所示的是图像 A 在新的灰度级空间[0, 255]内的新的灰度级。

表 13-15 图像 A 在[0, 255]上的新灰度级

灰度级	0	1	2	3	4	5	6	7
累计概率	0.18	0.37	0.49	0.59	0.71	0.78	0.84	1.00
均衡化值	47	94	125	151	182	198	213	255

经过均衡化处理后,图像 A 的灰度级在新灰度空间[0,255]内保持均衡。图 13-26 所示的是对图像 A 进行均衡化前的直方图,灰度级集中在[0, 7]之内。

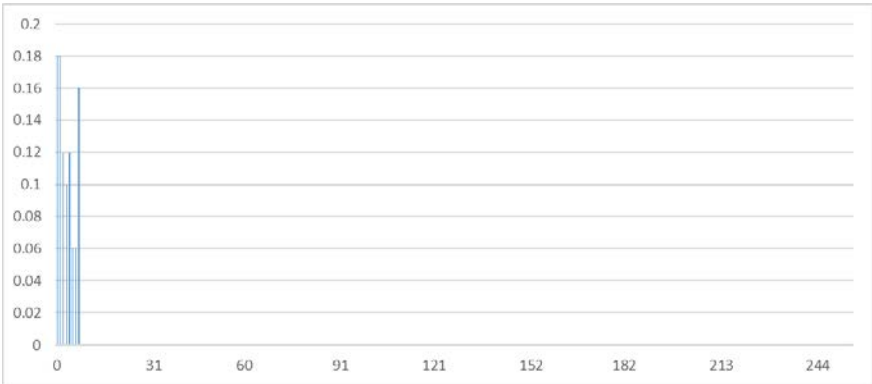


图 13-26 均衡化前的直方图

图 13-27 所示的是均衡化后的直方图，灰度级在整个灰度空间[0, 255]内分布得更均衡。

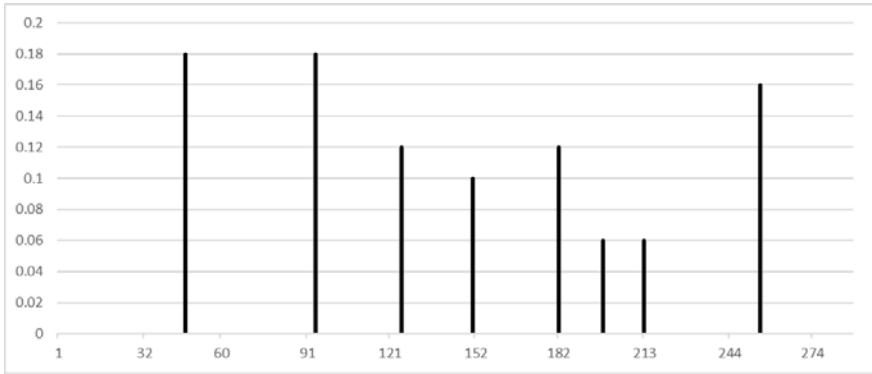


图 13-27 均衡化后的直方图

通过上述分析可知，通过如下两个步骤：

- (1) 计算累计直方图。
- (2) 将累计直方图进行区间转换。

可以让直方图达到均衡化的效果。

下面我们用数学表达式描述以上的直方图均衡化过程。假设图像中像素的总数是 N ，图像的灰度级数是 L ，灰度级空间是 $[0, L-1]$ ，用 n_k 表示第 k 级灰度（第 k 个灰度级，像素值为 k ）在图像内的像素点个数，那么该图像中灰度级为 r_k （第 k 个灰度级）出现的概率为：

$$P(r_k) = \frac{n_k}{N} \quad (k = 0, 1, \dots, L-1)$$

根据灰度级概率，对其进行均衡化处理的计算公式为：

$$s_k = T(r_k) = (L-1) \sum_{j=0}^k P(r_j) = (L-1) \sum_{j=0}^k \frac{n_j}{N} \quad (k = 0, 1, \dots, L-1)$$

式中， $\sum_{j=0}^k P(r_j)$ 表示累计概率，将该值与灰度级的最大值 $L-1$ 相乘即得到均衡化后的新灰度级（像素值）。

直方图均衡化使图像色彩更均衡、外观更清晰，也使图像更便于处理，它被广泛地应用在医学图像处理、车牌识别、人脸识别等领域。

13.3.2 直方图均衡化处理

OpenCV 使用函数 `cv2.equalizeHist()` 实现直方图均衡化。该函数的语法格式为：

```
dst = cv2.equalizeHist( src )
```

式中，`src` 是 8 位单通道原始图像，`dst` 是直方图均衡化处理的结果。

【例 13.11】 使用函数 `cv2.equalizeHist()` 实现直方图均衡化。

根据题目要求，编写代码如下：

```
#-----导入使用的模块-----
import cv2
import matplotlib.pyplot as plt
#-----读取原始图像-----
img = cv2.imread('image\\equ.bmp', cv2.IMREAD_GRAYSCALE)
#-----直方图均衡化处理-----
equ = cv2.equalizeHist(img)
#-----显示均衡化前后的图像-----
cv2.imshow("original",img)
cv2.imshow("result",equ)
#-----显示均衡化前后的直方图-----
plt.figure("原始图像直方图")    #构建窗口
plt.hist(img.ravel(),256)
plt.figure("均衡化结果直方图")  #构建新窗口
plt.hist(equ.ravel(),256)
#-----等待释放窗口-----
cv2.waitKey()
cv2.destroyAllWindows()
```

上述程序中，语句“`plt.figure("原始图像直方图")`”用于构造名为“原始图像直方图”的新窗口。运行程序，会显示如图 13-28 所示的图像。其中：

- (a)图是原始图像。
- (b)图是直方图均衡化后的图像。
- (c)图是原始图像的直方图。
- (d)图是经过直方图均衡化后的图像的直方图。



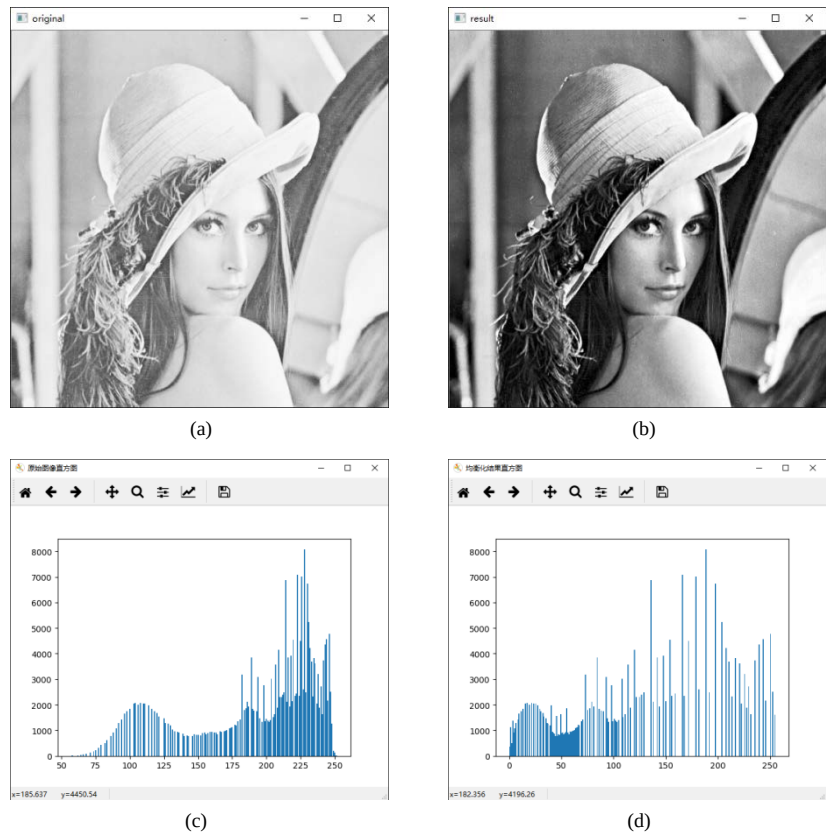


图 13-28 【例 13.11】程序的运行结果

在图 13-28 所示的处理结果中可以看到对比很明显：在直方图均衡化之前，图像整体比较亮；均衡化以后，图像的亮度变得比较均衡。而两幅图像的直方图的对比，则不太明显。这实际上体现了，均衡化是指综合考虑了统计概率和 HVS 的结果。

下面做简单的说明：

- 原始图像的直方图，大部分的像素值集中在右侧（线条密集）。这说明图像中位于[200, 255]区间的像素点很多，图像比较亮。
- 在均衡化后的直方图中，左侧的像素点比较密集而右侧的相对比较稀疏。但是，实际上人眼并不能明显感受到像素值的细微差别，所以我们可以将相近的像素值看成同一个像素值，这样就会得到类似于图 13-29 的直方图。此时，直方图内灰度级的分布就比较均衡了，是均衡一致的直方图。
- 或者从另外一个角度理解，在图 13-28 的(d)图中，左侧的线条密集，但是整体高度不高；右侧的线条稀疏，但是整体较高。这说明，较小的灰度级，每个灰度级上都有像素，但是个数较少；而较大的灰度级，在分布上比较稀疏，而且有一些灰度级并没有出现，但是出现了的灰度级其像素个数较多。如果将灰度级按照区间划分，则分布在每一个区间内的像素点个数大体是一致的。例如，对灰度级在[0, 50]、[51, 100]、[101,150]、[151, 200]、[201, 255]几个区间上进行划分，则每个区间内像素点的数量大体一致。所以，图 13-28 的(d)图是均衡化的直方图。

- 在图 13-28 的(d)图所示均衡化后的直方图中,左侧的像素点比较密集而右侧的相对比较稀疏。但是,实际上人眼并不能明显感受到像素值的细微差别,通常会将相近的像素值看成同一个像素值。例如, HVS 会主动将图 13-29 的(a)图中左侧的像素点取近似值,而得到图 13-29 的(b)图所示的直方图。同样的道理, HVS 人眼也会对图 13-29 的(b)图中右侧的像素值取均值近似处理,而得到如图 13-29 的(c)图所示的直方图。可以看到, HVS 对全部像素值取近似值后的直方图(图 13-29 的(c)图)内灰度级的分布是比较均匀的,是均衡一致的直方图。

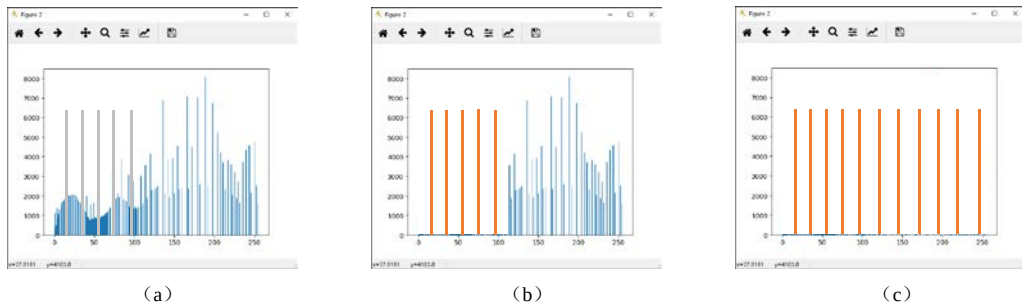


图 13-29 直方图说明

本例中的直方图均衡化进一步说明了,均衡化是指综合考虑了统计概率和 HVS 的均衡化结果。

13.4 pyplot 模块介绍

matplotlib.pyplot 模块¹提供了一个类似于 MATLAB 绘图方式的框架,可以使用其中的函数方便地绘制图形。

13.4.1 subplot 函数

模块 matplotlib.pyplot 提供了函数 matplotlib.pyplot.subplot() 用来向当前窗口内添加一个子窗口对象。该函数的语法格式为:

```
matplotlib.pyplot.subplot(nrows, ncols, index)
```

式中:

- `nrows` 为行数。
- `ncols` 为列数。
- `index` 为窗口序号。

例如, `subplot(2, 3, 4)` 表示在当前的两行三列的窗口的第 4 个位置上,添加 1 个子窗口,如图 13-30 所示。

1 为了使标题更简洁,故标题中未使用模块全称 `matplotlib.pyplot`。



图 13-30 添加子窗口（示意图）

需要注意的是，窗口是按照行方向排序的，而且序号是从“1”开始而不是从“0”开始的。如果所有参数都小于 10，可以省略彼此之间的逗号，直接写三个数字。例如，上述 subplot(2, 3, 4)可以直接表示为 subplot(234)。

【例 13.12】编写程序演示函数 subplot()的使用。

根据题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('image\\equ.bmp',cv2.IMREAD_GRAYSCALE)
equ = cv2.equalizeHist(img)
plt.figure("subplot 示例")
plt.subplot(121),plt.hist(img.ravel(),256)
plt.subplot(122),plt.hist(equ.ravel(),256)
```

运行上述程序，会显示如图 13-31 所示的图像。

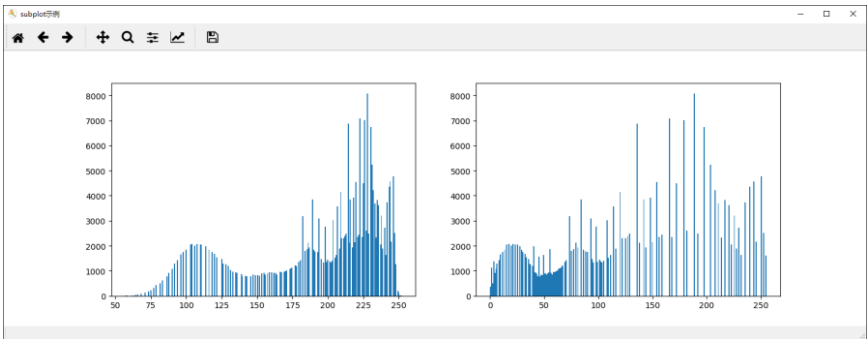


图 13-31 【例 13.12】程序的运行结果

13.4.2 imshow函数

模块 matplotlib.pyplot 提供了函数 matplotlib.pyplot.imshow()用来显示图像。其语法格式为：

```
matplotlib.pyplot.imshow(X, cmap=None)
```

式中：

- X 为图像信息，可以是各种形式的数值。
- cmap 表示色彩空间。该值是可选项，默认值为 null，默认使用 RGB(A)色彩空间。

【例 13.13】使用函数 matplotlib.pyplot.imshow()显示彩色图像。

根据题目的要求，编写代码如下：


```
import cv2
import matplotlib.pyplot as plt
img = cv2.imread('image\\girl.bmp')
imgRGB=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)
plt.figure("显示结果")
plt.subplot(121)
plt.imshow(img),plt.axis('off')
plt.subplot(122)
plt.imshow(imgRGB),plt.axis('off')
```

式中，语句“plt.axis('off')”表示关闭坐标轴的显示。

运行上述程序，会显示如图 13-32 所示的图像。

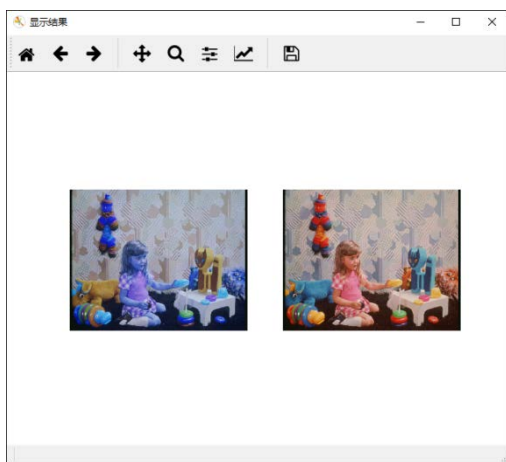


图 13-32 【例 13.13】程序的运行结果

在图 13-32 中，左图是直接使用默认色彩空间参数模式显示的彩色图像的结果，图像没有正常显示出来。这是因为通过函数 `cv2.imread()` 读取的图像，其通道顺序是 BGR 模式的。而函数 `matplotlib.pyplot.imshow()` 的显示顺序是 RGB 模式的，所以显示出来的图像通道顺序是错乱的，因而无法正常显示。

如果想使用函数 `matplotlib.pyplot.imshow()` 正常地显示函数 `cv2.imread()` 读取的图像，需要对读取的图像进行通道顺序转换。图 13-32 的右图是使用语句“`imgRGB=cv2.cvtColor(img,cv2.COLOR_BGR2RGB)`”将函数 `cv2.imread()` 读取到的图像 `img` 从 BGR 转换为 RGB 模式后，再使用函数 `matplotlib.pyplot.imshow()` 的默认色彩空间模式显示的彩色图像，可以看到图像显示正常。

【例 13.14】使用函数 `matplotlib.pyplot.imshow()` 显示灰度图像。

本例中，我们尝试使用不同的形式显示灰度图像，观察图像是否能够正常显示。

根据题目的要求，编写代码如下：

```
import cv2
import matplotlib.pyplot as plt
o = cv2.imread('image\\girl.bmp')
g=cv2.cvtColor(o, cv2.COLOR_BGR2GRAY)
```



```
plt.figure("灰度图像显示演示")
plt.subplot(221)
plt.imshow(o),plt.axis('off')
plt.subplot(222)
plt.imshow(o,cmap=plt.cm.gray),plt.axis('off')
plt.subplot(223)
plt.imshow(g),plt.axis('off')
plt.subplot(224)
plt.imshow(g,cmap=plt.cm.gray),plt.axis('off')
```

运行上述程序，会显示如图 13-33 所示的图像。可以看到，本例中所采用了几种针对灰度图像的显示方式，只有使用灰度图像作为参数，并且将色彩空间参数值设置为“`cmap=plt.cm.gray`”，灰度图像才被正常显示。因为印刷后无法观察到彩色效果，所以在图中对显示结果用文字进行了标注。

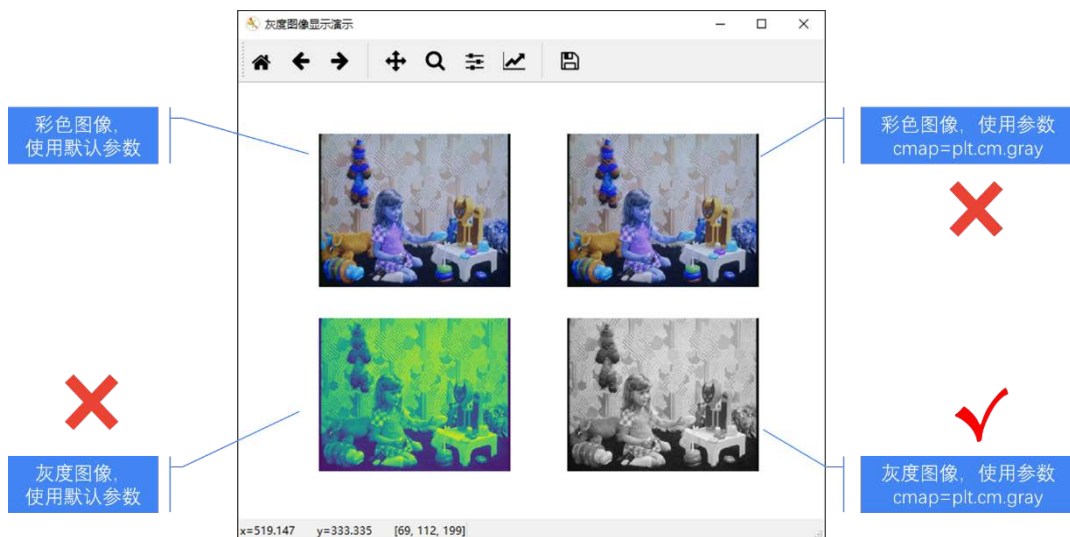


图 13-33 【例 13.14】程序的运行结果

本例以不同的形式展示了使用函数 `matplotlib.pyplot.imshow()` 显示灰度图像的效果。

- 左上角是原始图像。因为没有对读取的图像进行通道转换，直接将其作为彩色图像显示，所以显示结果中通道顺序是错乱的，图像没有正常显示。
- 右上角是我们尝试将彩色图像作为原始图像，将其色彩空间参数设置为“`cmap=plt.cm.gray`”，让彩色图像以灰度图像展示的结果，图像显示失败。
- 左下角是使用默认色彩空间参数，显示灰度图像的结果。这种情况下，灰度图像会采用默认的 RGB 模式显示。可以看到，图像没有正常显示。
- 右下角是将灰度图像作为原始图像，并将其色彩空间参数设置为“`cmap=plt.cm.gray`”而显示的灰度图像，图像显示正常。

【例 13.15】使用函数 `matplotlib.pyplot.imshow()` 以不同的参数形式显示灰度图像。

根据题目的要求，编写代码如下：

```
import cv2
```

```
import matplotlib.pyplot as plt
o = cv2.imread('image\\8.bmp')
g=cv2.cvtColor(o, cv2.COLOR_BGR2GRAY)
plt.figure("灰度图像显示演示")
plt.subplot(221); plt.imshow(g, cmap=plt.cm.gray)
plt.subplot(222); plt.imshow(g, cmap=plt.cm.gray_r)
plt.subplot(223); plt.imshow(g, cmap='gray')
plt.subplot(224); plt.imshow(g, cmap='gray_r')
```

上述代码中，色彩空间参数 `cmap` 的参数值 “`plt.cm.gray_r`” 及 “`gray_r`” 中的 “`r`” 是英文 “reverse” 的缩写，表示逆转的意思。

运行上述程序，会显示如图 13-34 所示的图像。

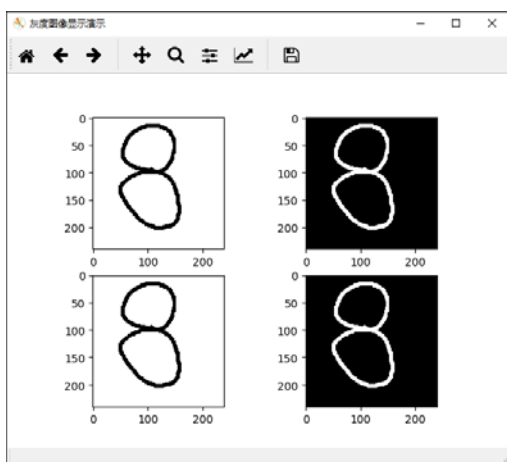


图 13-34 【例 13.15】程序的运行结果